

ABC450 F - Strongly Connected 2 题解

题意概括

图里固定存在一条反向链: $2 \rightarrow 1, 3 \rightarrow 2, \dots, N \rightarrow N - 1$ 。另外还有 M 条可删边, 第 i 条是 $X_i \rightarrow Y_i$, 并且 $X_i < Y_i$ 。

我们要从这 M 条可删边里删掉若干条, 统计删完以后整张图仍然强连通的方案数。

解题思路

先看强连通的判定。

由于固定边保证了“从大点往小点”一定能走, 所以真正缺的是“从小点往大点”的能力。把顶点分成前缀 $\{1, 2, \dots, k\}$ 和后缀 $\{k + 1, \dots, N\}$, 如果删边后不存在一条可删边满足 $X_i \leq k < Y_i$, 那么我们就无法从前缀走到后缀, 图一定不强连通。

反过来, 如果对每个 $k = 1, 2, \dots, N - 1$, 都至少保留了一条满足 $X_i \leq k < Y_i$ 的边, 那么从 1 出发就能不断跨过每一道分界线, 再配合反向链回退到任意更小的点, 于是整张图强连通。

所以原问题等价于:

- 把每条可删边 $X_i \rightarrow Y_i$ 看成区间 $[X_i, Y_i - 1]$;
- 统计有多少个保留下来的区间集合能覆盖所有位置 $1, 2, \dots, N - 1$ 。

接下来按左端点从小到大处理这些区间。

设当前已经处理完所有左端点不超过 i 的区间, 记 mx 为“从 1 出发, 目前最远能到达的顶点编号”。如果 $mx > i$, 说明前 i 道分界线都已经被覆盖; 如果 $mx \leq i$, 那第 i 道分界线就过不去。

直接做 $dp[i][mx]$ 会是 $O(N^2)$, 需要换个写法。

定义后缀和状态 $F(r)$:

- $F(r)$ 表示当前合法方案中, 满足 $mx \geq r$ 的方案数。

再固定当前左端点 i , 设:

- tot_i 为起点恰好等于 i 的边数;
- $T_i = 2^{tot_i}$, 表示这些边任意保留或删除的总方案数;
- $C_i(r)$ 表示“只保留终点 $< r$ 的边”的方案数。

如果起点为 i 的边里有 $cnt_i(r)$ 条终点 $< r$, 那么显然有 $C_i(r) = 2^{cnt_i(r)}$ 。

现在考虑更新后的 $F'(r)$:

- 若更新前已经有 $mx \geq r$, 那么这批起点为 i 的边怎么选都不会让“能到达 r ”这件事失效, 共有 T_i 种选法;
- 若更新前 $mx < r$, 那就必须至少保留一条终点 $\geq r$ 的边, 才能把 mx 推到 r 及以上, 这样的选法有 $T_i - C_i(r)$ 种。

于是可得转移:

$$F'(r) = C_i(r) * F(r) + (T_i - C_i(r)) * S$$

其中 $S = F(1)$ 是更新前所有合法方案总数。

这个式子有两个关键好处:

1. 对固定的 i , 它对每个 r 都只是一次仿射变换;
2. $C_i(r)$ 只会在 r 越过某个终点时发生变化。

因此, 把同一个起点 i 的所有终点排序后, r 会被切成若干连续区间; 在每一段上, $C_i(r)$ 都是常数, 我们就可以在线段树上做一次区间仿射更新。

这里参考实现保留了 `vector<int> to[i]` 这个 STL 容器。原因是每个起点的出边条数事先不固定, 而且我们需要把同一起点的所有终点集中起来排序、分段处理; 用 `vector` 分组会比手写链表桶或多组偏移数组更直观, 也更适合教学说明。

每处理完一个起点 i 之后, 还要保证第 i 道分界线已经被覆盖, 也就是必须满足 $mx \geq i + 1$ 。因此新的总合法方案数就是 $F(i + 1)$, 而所有 $mx \leq i$ 的状态都要丢掉。对后缀和数组来说, 这等于:

- 令 $S' = F(i + 1)$;
- 把所有 $r \leq i + 1$ 的位置都直接改成 S' 。

这样一路处理到 $i = N - 1$, 最后的答案就是 $F(1)$ 。

正确性说明

先证明强连通判定的等价性。

若存在某个 k , 删边后没有任何一条可删边满足 $X_i \leq k < Y_i$, 那么所有从前缀 $\{1, 2, \dots, k\}$ 出发的可删边仍然只能落在前缀内部, 我们不可能从前缀走到后缀, 所以图不强连通。

若对每个 k 都至少保留了一条满足 $X_i \leq k < Y_i$ 的边, 那么从 1 开始, 只要当前已经能到达某个前缀 $\{1, 2, \dots, p\}$, 就一定存在一条边从这个前缀连到更大的顶点, 于是可达前缀会继续向右扩张。最终可以到达 N , 再利用固定存在的反向链, 就能从任意大点走向任意小点, 因此整张图强连通。

再证明 DP 转移正确。

固定某个起点 i 和某个阈值 r 。

如果更新前已经满足 $mx \geq r$ ，说明不使用起点为 i 的任何边，也已经能从 1 到达 r 及以上。此时这批边的选择不会破坏这个事实，所以一共有 T_i 种可选方案。

如果更新前 $mx < r$ ，那么仅靠旧状态还到不了 r 。要让更新后满足 $mx \geq r$ ，就必须保留至少一条终点 $\geq r$ 的边。所有子集总共有 T_i 个，其中“所有保留边终点都 $< r$ ”的坏方案有 $C_i(r)$ 个，因此好方案数恰好是 $T_i - C_i(r)$ 。

于是 $F'(r) = C_i(r) * F(r) + (T_i - C_i(r)) * S$ 成立。

最后说明“把 $r \leq i + 1$ 全部赋成 $F(i + 1)$ ”为什么正确。处理完起点 i 后，合法状态必须满足 $mx \geq i + 1$ ，也就是所有 $mx \leq i$ 的状态全部无效。对于任何 $r \leq i + 1$ ，合法状态里“ $mx \geq r$ ”与“任意合法状态”是同一件事，所以对应的后缀和都应当等于新的总方案数 $F(i + 1)$ 。

因此，整套算法始终精确统计了覆盖所有分界线的区间子集数量，也就得到了原题答案。

复杂度

- 时间复杂度： $O((N + M) \log N)$
- 空间复杂度： $O(N + M)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;

const int mod = 998244353;
const int maxn = 200000;

int n, m;
int pw2[maxn + 5];
int leaf_value[(maxn + 5) << 2];
int lazy_mul[(maxn + 5) << 2];
int lazy_add[(maxn + 5) << 2];
int lazy_set[(maxn + 5) << 2];
vector<int> to[maxn + 5];

int add_mod(int a, int b){
    a += b;
    if(a >= mod){
        a -= mod;
    }
}
```

```
}  
    return a;  
}  
  
int sub_mod(int a, int b){  
    a -= b;  
    if(a < 0){  
        a += mod;  
    }  
    return a;  
}  
  
int mul_mod(long long a, long long b){  
    return (int)(a * b % mod);  
}
```

```
void build(int idx, int l, int r){  
    lazy_mul[idx] = 1;  
    lazy_add[idx] = 0;  
    lazy_set[idx] = -1;  
    if(l == r){  
        if(l == 1){  
            leaf_value[idx] = 1;  
        } else {  
            leaf_value[idx] = 0;  
        }  
        return;  
    }  
    int mid = (l + r) >> 1;  
    build(idx << 1, l, mid);  
    build(idx << 1 | 1, mid + 1, r);  
}
```

```
void apply_set(int idx, int l, int r, int value){  
    if(l == r){  
        leaf_value[idx] = value;  
        return;  
    }  
    lazy_set[idx] = value;  
    lazy_mul[idx] = 1;  
    lazy_add[idx] = 0;
```

```

}

void apply_affine(int idx, int l, int r, int mul_value, int add_value){
    if(l == r){
        leaf_value[idx] = add_mod(mul_mod(leaf_value[idx], mul_value),
                                   add_value);
        return;
    }
    if(lazy_set[idx] != -1){
        lazy_set[idx] = add_mod(mul_mod(lazy_set[idx], mul_value), add_value);
        return;
    }
    lazy_mul[idx] = mul_mod(lazy_mul[idx], mul_value);
    lazy_add[idx] = add_mod(mul_mod(lazy_add[idx], mul_value), add_value);
}

void push_down(int idx, int l, int r){
    if(l == r){
        return;
    }
    int mid = (l + r) >> 1;
    if(lazy_set[idx] != -1){
        apply_set(idx << 1, l, mid, lazy_set[idx]);
        apply_set(idx << 1 | 1, mid + 1, r, lazy_set[idx]);
        lazy_set[idx] = -1;
    }
    if(lazy_mul[idx] != 1 || lazy_add[idx] != 0){
        apply_affine(idx << 1, l, mid, lazy_mul[idx], lazy_add[idx]);
        apply_affine(idx << 1 | 1, mid + 1, r, lazy_mul[idx], lazy_add[idx]);
        lazy_mul[idx] = 1;
        lazy_add[idx] = 0;
    }
}

void range_set(int idx, int l, int r, int ql, int qr, int value){
    if(ql <= l && r <= qr){
        apply_set(idx, l, r, value);
        return;
    }
    push_down(idx, l, r);
    int mid = (l + r) >> 1;

```

```

    if(q1 <= mid){
        range_set(idx << 1, l, mid, q1, qr, value);
    }
    if(qr > mid){
        range_set(idx << 1 | 1, mid + 1, r, q1, qr, value);
    }
}

void range_affine(int idx, int l, int r, int q1, int qr, int mul_value, int
    add_value){
    if(q1 <= l && r <= qr){
        apply_affine(idx, l, r, mul_value, add_value);
        return;
    }
    push_down(idx, l, r);
    int mid = (l + r) >> 1;
    if(q1 <= mid){
        range_affine(idx << 1, l, mid, q1, qr, mul_value, add_value);
    }
    if(qr > mid){
        range_affine(idx << 1 | 1, mid + 1, r, q1, qr, mul_value, add_value);
    }
}

int query_point(int idx, int l, int r, int pos){
    if(l == r){
        return leaf_value[idx];
    }
    push_down(idx, l, r);
    int mid = (l + r) >> 1;
    if(pos <= mid){
        return query_point(idx << 1, l, mid, pos);
    }
    return query_point(idx << 1 | 1, mid + 1, r, pos);
}

int main(){
    cin >> n >> m;
    for(int i=1; i<=m; i++){
        int x, y;
        cin >> x >> y;
    }
}

```

```

        to[x].push_back(y);
    }

    pw2[0] = 1;
    for(int i=1; i<=m; i++){
        pw2[i] = add_mod(pw2[i - 1], pw2[i - 1]);
    }

    for(int i=1; i<=n; i++){
        sort(to[i].begin(), to[i].end());
    }

    build(1, 1, n);

    for(int i=1; i<=n - 1; i++){
        // S = F(1), 表示处理当前起点之前的所有合法方案数
        int sum = query_point(1, 1, n, 1);
        int deg = (int)to[i].size();

        // 按终点分段, 在线段树上对每一段 r 做
        //  $F'(r) = C_i(r) * F(r) + (T_i - C_i(r)) * S$ 
        int pos = 1;
        int seen = 0;
        for(int j=0; j<deg; ){
            int y = to[i][j];
            if(pos <= y){
                int keep_small = pw2[seen];
                int add_value = mul_mod(sub_mod(pw2[deg], keep_small), sum);
                range_affine(1, 1, n, pos, y, keep_small, add_value);
            }
            while(j < deg && to[i][j] == y){
                seen++;
                j++;
            }
            pos = y + 1;
        }
        if(pos <= n){
            int keep_small = pw2[seen];
            int add_value = mul_mod(sub_mod(pw2[deg], keep_small), sum);
            range_affine(1, 1, n, pos, n, keep_small, add_value);
        }
    }
}

```

```
// 处理完起点 i 后, 必须满足  $mx \geq i + 1$   
int new_sum = query_point(1, 1, n, i + 1);  
range_set(1, 1, n, 1, i + 1, new_sum);  
}  
  
cout << query_point(1, 1, n, 1) << '\n';  
  
return 0;  
}
```